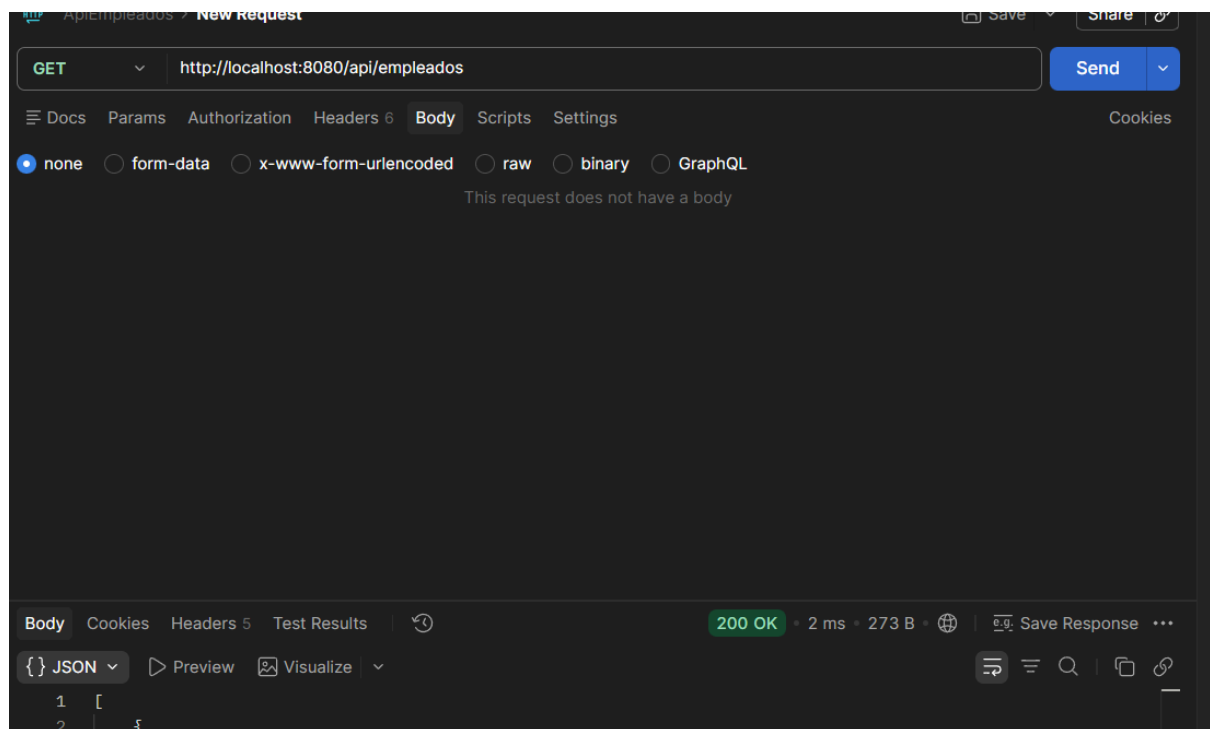
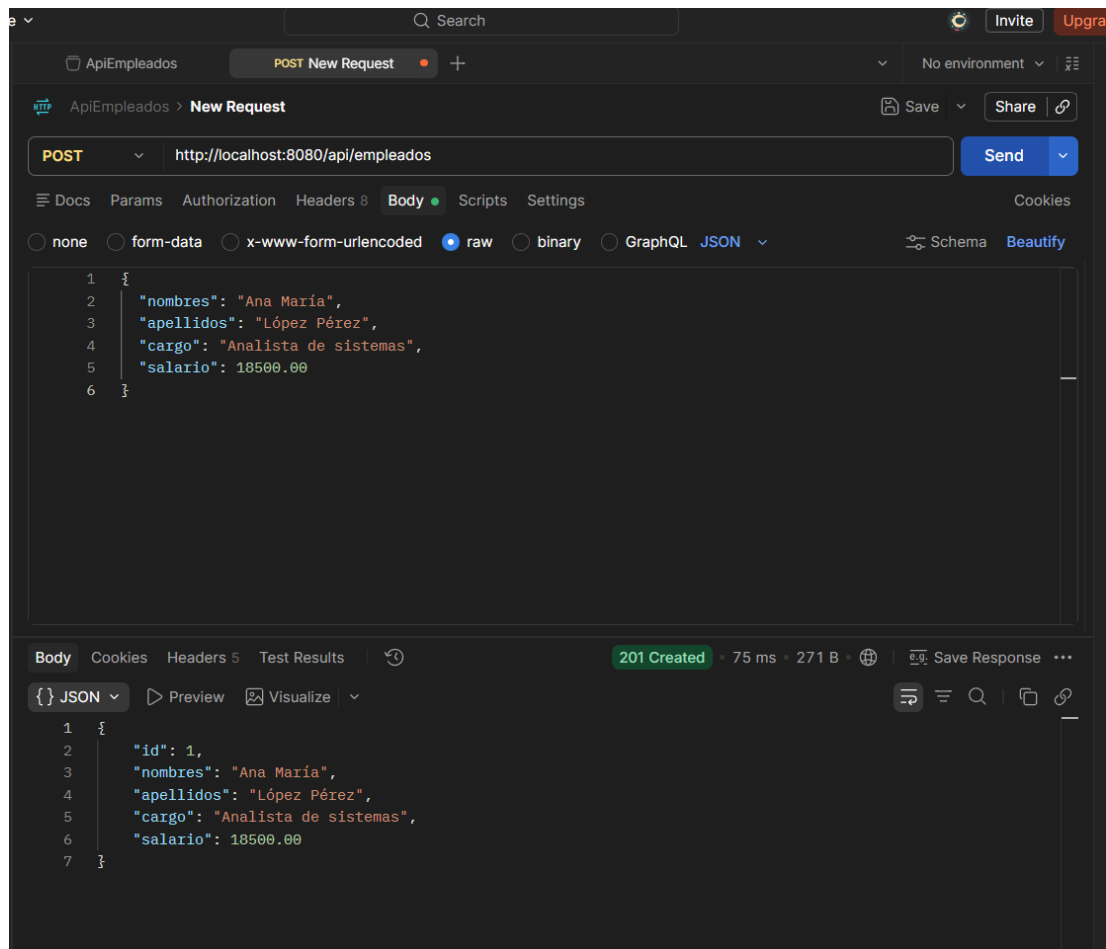




ApiEmpleados > New Request

SaveShare

GEThttp://localhost:8080/api/empleados/1Send

DocsParamsAuthorizationHeaders 6BodyScriptsSettingsCookies

☒ none

☐ form-data

☐ x-www-form-urlencoded

☐ raw

☐ binary

☐ GraphQL

This request does not have a body

BodyCookiesHeaders 5Test Results

200 OK6 ms271 BSave Response

{ } JSON

PreviewVisualize

```
1 {
2   "id": 1,
3   "nombres": "Ana María",
4   "apellidos": "López Pérez",
5   "cargo": "Analista de sistemas",
6   "salario": 18500.00
7 }
```

ApiEmpleados > New Request

GET

http://localhost:8080/api/empleados/999

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

This request does not have a body

Body

Cookies

Headers 5

Test Results

404 Not Found

7 ms

198 B

Save Response

JSON

Preview

Debug with AI

1 {

2 | "mensaje": "Empleado no encontrado"

3 }

HTTP

ApiEmpleados > New Request

Save

Share

PUT

http://localhost:8080/api/empleados/1

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

```
1 {
2   "nombres": "Ana María",
3   "apellidos": "López Pérez",
4   "cargo": "Desarrolladora Backend",
5   "salario": 25000.00
6 }
```

Body

Cookies

Headers 5

Test Results

200 OK

6 ms

273 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "id": 1,
3   "nombres": "Ana María",
4   "apellidos": "López Pérez",
5   "cargo": "Desarrolladora Backend",
6   "salario": 25000.00
7 }
```

API

ApiEmpleados > New Request

Save

Share

POST

http://localhost:8080/api/empleados

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Schema

Beautify

```
1 {
2   "nombres": "",
3   "apellidos": "Pérez",
4   "cargo": "",
5   "salario": 0
6 }
```

Body

Cookies

Headers 4

Test Results

400 Bad Request

6 ms

344 B

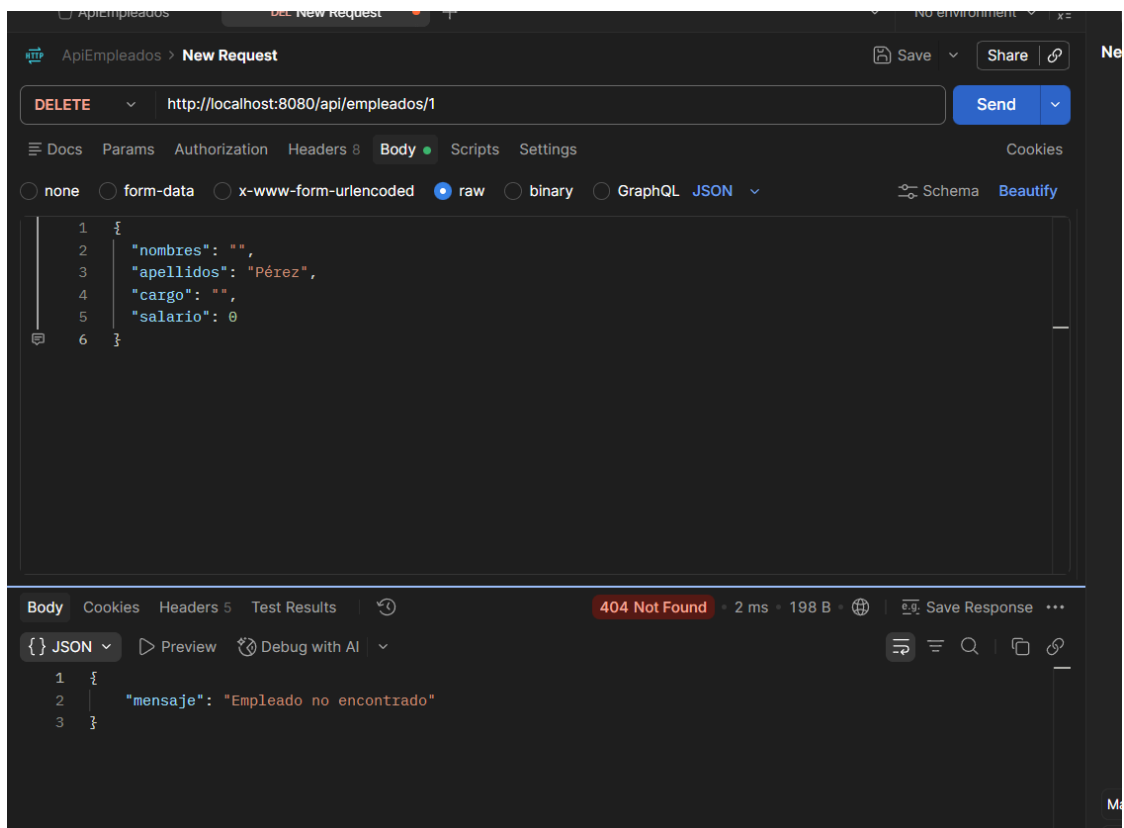
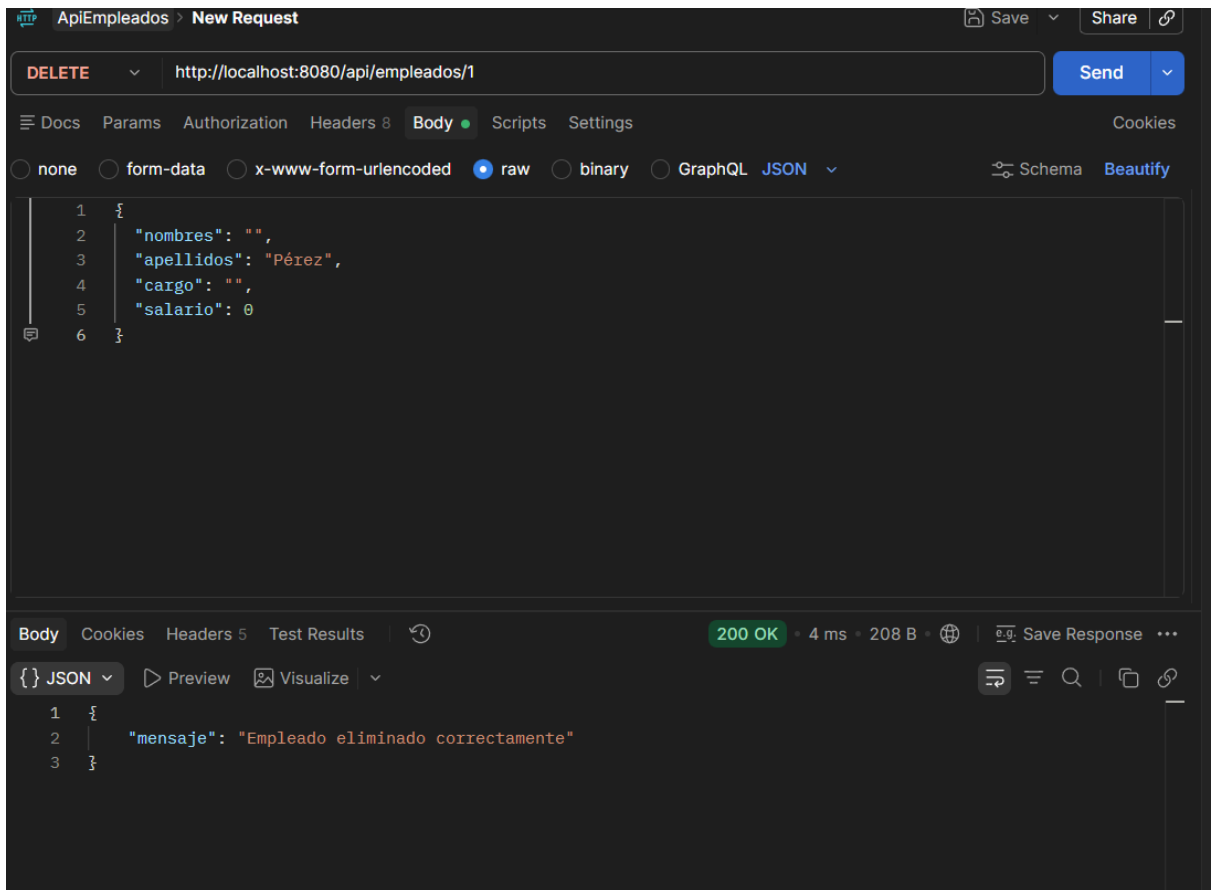
Save Response

{ } JSON

Preview

Debug with AI

```
1 {
2   "estado": 400,
3   "mensaje": "Los datos enviados no son válidos",
4   "errores": {
5     "nombres": "Los nombres deben tener entre 2 y 60 caracteres",
6     "cargo": "El cargo es obligatorio",
7     "salario": "El salario debe ser mayor que 0"
8   }
9 }
```



1. ¿Qué diferencia existe entre `@Controller` y `@RestController`?

La principal diferencia es que `@Controller` se utiliza normalmente para manejar solicitudes y devolver vistas, mientras que `@RestController` se utiliza para crear servicios REST y devolver directamente datos en el cuerpo de la respuesta, generalmente en formato JSON.

2. ¿Qué función cumple `@RequestBody`?

La anotación `@RequestBody` permite recibir los datos enviados por el cliente en el cuerpo de una solicitud HTTP y convertirlos automáticamente en un objeto Java, como un DTO.

3. ¿Por qué se utiliza `@Valid` junto al DTO?

`@Valid` se utiliza para indicar a Spring que debe validar los datos recibidos según las reglas definidas en el DTO, como campos obligatorios, tamaños mínimos o máximos y valores permitidos.

4. ¿Qué código HTTP debe devolver un registro creado correctamente?

Cuando un registro se crea correctamente, la API debe devolver el código HTTP 201 Created, ya que este indica que el recurso fue creado exitosamente.

5. ¿Cómo interviene Jackson en las respuestas de la API?

Jackson se encarga de convertir automáticamente los objetos Java en formato JSON cuando la API envía una respuesta. También realiza el proceso contrario, convirtiendo el JSON recibido en una solicitud en objetos Java que pueden ser utilizados por la aplicación.